

Three Figures

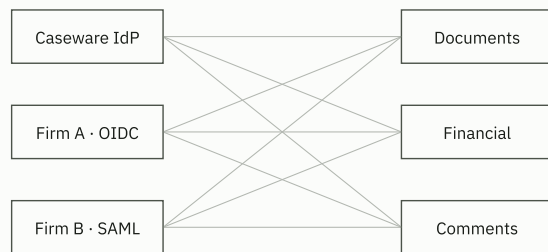
Companion to the design document. Each figure shows a mechanism the prose would otherwise have to be assembled from — the boundary being crossed, the hop being added, the data that moves.

§1A Federation Broker

Why Collaborate runs its own authorization server rather than letting clients take tokens directly from each firm's identity provider.

REJECTED · DIRECT FEDERATION

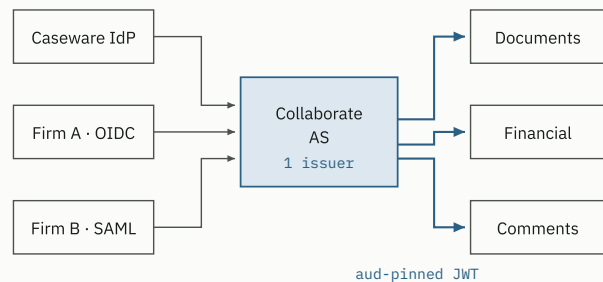
9 trust relationships · grows with every firm



Every service trusts every issuer.
SAML emits an XML assertion, not an access token — unusable as-is.

CHOSEN · FEDERATION BROKER

6 relationships · one issuer to trust



Upstream protocols terminate here. SAML is translated once, not three times.

aud-pinned JWT

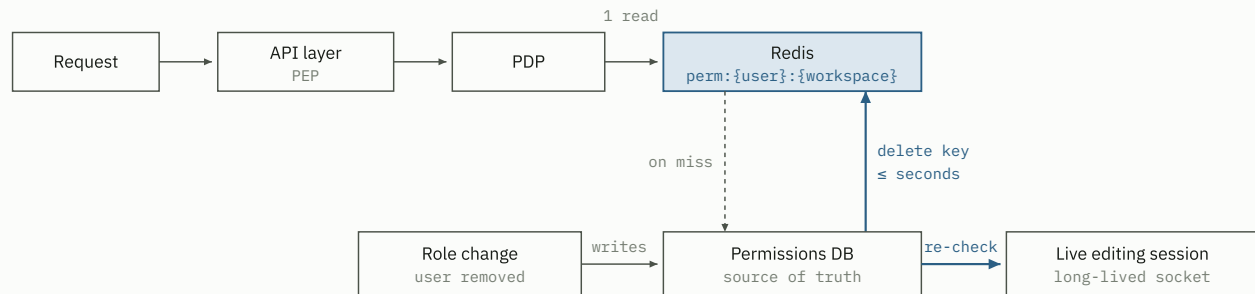
FIGURE 1 · TRUST TOPOLOGY

Direct federation makes every downstream service trust every identity provider, and the count grows with each firm onboarded. The broker terminates all upstream protocols at one point, so resource services validate exactly one issuer. The SAML case decides it outright: an XML assertion is not an access token, so a translation step is required either way — the only question is whether it happens once or once per service.

§1B Policy Decision Point

Where authorization reads resolve, and how a permission change reaches both the cache and an open collaborative session.

HOT PATH · NO DATABASE



The request path never reaches the database. Change events travel the other way – outward from the source of truth.

FIGURE 2 · READ PATH AND REVOCATION LOOP

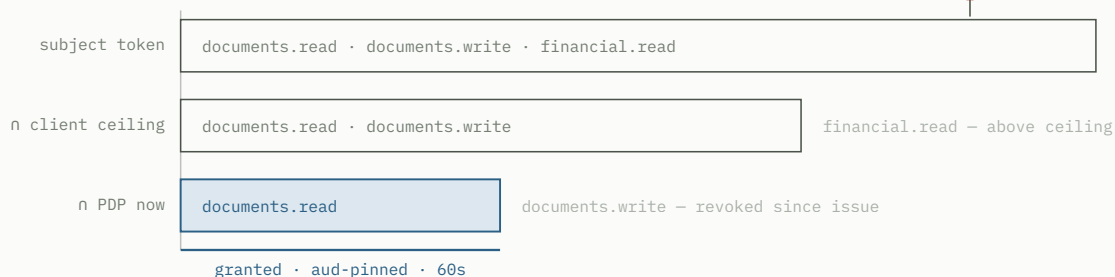
Authorization reads resolve against Redis; the database is touched only on a rare miss. Revocation runs in the opposite direction: a permission change writes to the database, which emits an event that deletes the cached key *and* reaches long-lived sockets directly. That second arrow matters — after the handshake, an open editing session is never seen by the request path again, so request-path enforcement alone would leave it authorized indefinitely.

§1C On-Behalf-Of Exchange

How a delegated token's scope is computed, and why it can only ever shrink.

SCOPE NARROWING · INTERSECTION, NEVER UNION

above every ceiling → `invalid_scope`



Each row is a ceiling. The issued token is the intersection of all of them, so no path through this pipeline can widen a caller's authority.

FIGURE 3 · SCOPE NARROWING

Granted scope is computed as an intersection, so it can only shrink. The two failure paths are treated differently on purpose: a scope the subject token never carried is an escalation attempt and fails loudly with `invalid_scope`, while a scope the user has *since lost* is dropped silently and reported back in the response. The first is an attack signal worth surfacing; the second is ordinary staleness that should not break a legitimate integration.